78. Многомерные массивы

Элементы массива могут быть любого типа данных, даже **массива**!

Оглавление:

1. Многомерные массивы
2. Инициализация двумерных массивов
3. Доступ к элементам в двумерном массиве
4. Многомерные массивы более двух измерений
5. Пример двумерного массива

Многомерные массивы

Массив массивов называется **многомерным массивом**:

```
int array[2][4]; // 2-элементный массив из 4-элементных массивов
```

Поскольку у нас есть 2 индекса, то это **двумерный массив**.

В двумерном массиве первый (левый) индекс принято читать как количество строк, а второй (правый) как количество столбцов. Массив выше можно представить следующим образом:

```
[0][0]  [0][1]  [0][2]  [0][3]  // строка №0
[1][0]  [1][1]  [1][2]  [1][3]  // строка №1
```

Чтобы получить доступ к элементам двумерного массива, просто используйте два индекса:

```
array[1][3] = 7; // без приставки int (типа данных)
```

Инициализация двумерных массивов

Для инициализации двумерного массива проще всего использовать вложенные фигурные скобки, где каждый набор значений соответствует определенной строке:

```
int array[3][5] =
{
{ 1, 2, 3, 4, 5 }, // строка №0
{ 6, 7, 8, 9, 10 }, // строка №1
{ 11, 12, 13, 14, 15 } // строка №2
};
```

Хотя некоторые компиляторы могут позволить вам упустить внутренние фигурные скобки, все же рекомендуется указывать их в любом случае: улучшается читабельность и уменьшается вероятность получения незапланированных нулевых элементов массива из-за того, что C++ заменяет отсутствующие инициализаторы значением `0`:

```
int array[3][5] =
{
{ 2, 4  }, // строка №0 = 2, 4, 0, 0, 0
{ 1, 3, 7 }, // строка №1 = 1, 3, 7, 0, 0
{ 8, 9, 11, 12 } // строка №2 = 8, 9, 11, 12, 0
};
```

В двумерном массиве со **списком инициализаторов** можно не указывать только левый индекс (длину массива):

```
int array[][5] =
{
{ 1, 2, 3, 4, 5 },
{ 6, 7, 8, 9, 10 },
{ 11, 12, 13, 14, 15 }
};
```

Компилятор может сам вычислить количество строк в массиве. Однако не указывать два индекса — это уже ошибка:

```
int array[][] =
{
{ 3, 4, 7, 8 },
{ 1, 2, 6, 9 }
};
```

Подобно обычным массивам, многомерные массивы можно инициализировать значением `0` следующим образом:

```
int array[3][5] = { 0 };
```

Обратите внимание, это работает только в том случае, если вы явно объявляете длину массива (указываете левый индекс)! В противном случае, вы получите двумерный массив с 1 строкой.

Доступ к элементам в двумерном массиве

Для доступа ко всем элементам двумерного массива требуется два цикла: один для строк и один для столбцов. Поскольку доступ к двумерным массивам обычно выполняется по строкам, то левый индекс используется в качестве внешнего цикла:

```
for (int row = 0; row < numRows; ++row) // доступ по строкам
    for (int col = 0; col < numCols; ++col) // доступ к каждому элементу в строке
        std::cout << array[row][col];
```

Многомерные массивы более двух измерений

Многомерные массивы могут быть более двух измерений. Например, объявление трехмерного массива:

```
int array[4][3][2];
```

Трехмерные массивы трудно инициализировать любым интуитивным способом с использованием списка инициализаторов, поэтому лучше инициализировать весь массив значением `0` и явно присваивать элементам значения с помощью вложенных циклов.

Доступ к элементам трехмерного массива осуществляется так же, как и к элементам двумерного массива:

```
std::cout << array[3][2][1];
```

Пример двумерного массива

Рассмотрим пример использования двумерного массива:

```
#include <iostream>

int main()
{
    // Объявляем массив 10x10
    const int numRows = 10;
    const int numCols = 10;
    int product[numRows][numCols] = { 0 };

    // Создаем таблицу умножения
    for (int row = 0; row < numRows; ++row)
        for (int col = 0; col < numCols; ++col)
            product[row][col] = row * col;

    // Выводим таблицу умножения
    for (int row = 1; row < numRows; ++row)
    {
        for (int col = 1; col < numCols; ++col)
            std::cout << product[row][col] << "\t";

        std::cout << '\n';
    }

    return 0;
}
```

Эта программа вычисляет и выводит таблицу умножения от 1 до 9 (включительно). Обратите внимание, при выводе таблицы в **цикле for** мы начинаем с 1 вместо 0. Это делается с целью предотвращения вывода нулевой строки и нулевого столбца, содержащих одни нули!

Результат выполнения программы:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Двумерные массивы обычно используются в играх типа *tile-based*, где каждый элемент массива представляет собой один фрагмент/плитку. Они также используются в компьютерной 3D-графике (в виде *матриц*) для вращения, масштабирования и отражения фигур.